19 de diciembre de 2024

JFlex Lexer

Ingeniería en Computación

Compiladores e Intérpretes

Autores:
Danilo Duque
Emmanuel Rojas

Profesor:
Allan Rodríguez

Manual de usuario

Este manual describe los pasos necesarios para compilar y ejecutar el programa desde el directorio principal del proyecto.

CUP/JFlex

Para compilar el componente CUP, se debe ejecutar el siguiente comando en la terminal, asegurándose de sustituir `/ruta/al/java-cup-x.jar` por la ubicación real del archivo `java-cup-x.jar` en su sistema:

```
java -jar /ruta/al/java-cup-x.jar -parser
```

Este comando generará dos archivos: uno llamado `Parser` y otro llamado `sym`. En este proyecto, solo se utiliza el archivo `sym`, por lo que no es necesario interactuar con `Parser`.

Una vez generado el archivo `sym`, se puede compilar el lexer escribiendo el siguiente comando en la terminal. Nuevamente, reemplace `/ruta/al/jflex.jar` por la ubicación real de `jflex.jar`:

```
java -jar /ruta/al/jflex.jar lexer.flex
```

Este comando compilará el archivo `lexer.flex`, generando el código correspondiente al lexer.

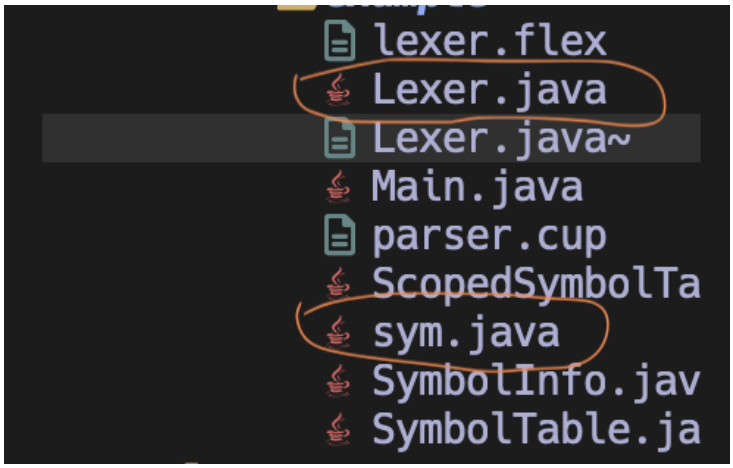

Figura 1: Archivos generados por CUP y JFlex

Ejecución del programa

Después de haber generado los archivos de CUP y JFlex, podemos crear el jar del programa con la línea `./gradlew clean build` esto generará un archivo en el directorio libs dentro de build, luego de esto podemos utilizar el comando:

```
java -jar build/libs/P1-1.0-SNAPSHOT.jar archivo.txt
```

el archivo.txt puede ser cambiado por un nombre de archivo diferente, este archivo debe estar en el directorio:

```
src/main/java/resources
```

Funcionamiento

Cuando corremos la línea para generar el CUP vampos a encontrar el siguiente mensaje

```
------- CUP v0.11b 20160615 (GIT 4ac7450) Parser Generation Summary -------
  0 errors and 49 warnings
  54 terminals, 2 non-terminals, and 5 productions declared,
  producing 7 unique parse states.
  49 terminals declared but not used.
  0 non-terminals declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "Parser.java", and "Sym.java".
```

Figura 2: Mensaje al generar el sym

Mensaje completo

Desglose del mensaje

- **0 errors and 49 warnings:** - No hubo errores, pero hay 49 advertencias. Estas advierten que hay elementos declarados que no se están utilizando, como terminales o producciones innecesarias.
- **54 terminals, 2 non-terminals, and 5 productions declared:** - Se declararon 54 terminales (símbolos atómicos del lexer), 2 no-terminales (estructuras derivadas) y 5 producciones (reglas gramaticales).
- **Producing 7 unique parse states:** - Se generaron 7 estados de análisis sintáctico únicos, necesarios para manejar las reglas gramaticales definidas.
- **49 terminals declared but not used:** - De los 54 terminales declarados, 49 no están siendo utilizados en las producciones.
- **0 non-terminals declared but not used:** - Todos los no-terminales definidos se están utilizando correctamente.
- **0 productions never reduced:** - Todas las producciones se utilizan durante el análisis sintáctico, lo que indica que la gramática es consistente.

- **0 conflicts detected (0 expected):** - No se detectaron conflictos (como conflictos de desplazamiento/reducción o reducción/reducción), lo que significa que la gramática es bien definida y no ambigua.
- **Code written to "Parser.java", and "Sym.java":** - CUP generó dos archivos:
  - `Parser.java`: Contiene el código del parser, que procesa el flujo de tokens y verifica si cumple con la gramática.
  - `Sym.java`: Define los terminales utilizados por el lexer y el parser.

Luego cuando ya se genera el CUP podemos correr el Lexer con la línea anted mencionada

Mensaje completo

```
306 states before minimization, 296 states in minimized
   DFA
Old file "src/main/java/org/example/Lexer.java" saved as
   "src/main/java/org/example/Lexer.java~"
Writing code to "src/main/java/org/example/Lexer.java"
```

Desglose del mensaje

- **306 states before minimization, 296 states in minimized DFA:** JFlex generó un Autómata Finito Determinista (DFA) con 306 estados. Luego, aplicó un proceso de minimización para reducirlos a 296, eliminando estados redundantes y optimizando el DFA.
- **Old file "src/main/java/org/example/Lexer.java"saved as "src/main/java/org/example/Lexer.java ":** Si existía un archivo llamado `Lexer.java`, JFlex creó una copia de respaldo (`Lexer.java`) para proteger el archivo original.
- **Writing code to "src/main/java/org/example/Lexer.java":** JFlex escribió el código generado del lexer en el archivo `Lexer.java`, el cual contiene el DFA en forma de código Java.

Luego de haber verificado que estás dos fases hayan sido hechas correctamente, podemos correr el programa principal, que va a generar el archivo con la tabla de símbolos, esta tabla se puede verificar en output.txt

1. Descripción del Problema

La configuración de chips en sistemas empotrados es un área que enfrenta desafíos crecientes debido a la constante evolución tecnológica y la necesidad de herramientas ligeras y potentes. Los lenguajes de programación tradicionales, a menudo, no cumplen con los requisitos de eficiencia y simplicidad necesarios para este tipo de aplicaciones. Por esta razón, un grupo de desarrolladores ha propuesto el desarrollo de un nuevo lenguaje de programación imperativo, diseñado específicamente para abordar estas necesidades.

El objetivo principal de este proyecto es implementar la fase de análisis léxico para este lenguaje, asegurando un reconocimiento eficiente de los elementos del código fuente, como tokens, estructuras sintácticas y la detección de errores léxicos. Este análisis es fundamental para garantizar que el lenguaje pueda ser procesado correctamente por las siguientes fases del compilador.

El problema central radica en diseñar un analizador léxico que cumpla con las siguientes especificaciones:

- Reconocer y clasificar los tokens definidos en la gramática del lenguaje.
- Detectar errores léxicos y manejar su recuperación mediante la técnica de Recuperación en Modo Pánico".
- Generar una representación estructurada del código fuente que facilite las etapas posteriores del análisis sintáctico y semántico.

Este proyecto se desarrollará utilizando las herramientas JFlex y Cup, las cuales permiten generar automáticamente los analizadores léxicos y sintácticos, respetando la gramática BNF descrita. Asimismo, se espera que el sistema sea capaz de procesar archivos fuente, generando un reporte detallado de los tokens y errores encontrados, con información como tipo, lexema, línea y columna de aparición.

La correcta implementación del análisis léxico no solo es crucial para validar la viabilidad del lenguaje propuesto, sino también para garantizar un desarrollo eficiente y confiable de sistemas empotrados en el futuro.

2. Diseño del Programa

El diseño del programa se centra en la implementación de un analizador léxico para el lenguaje definido en la gramática BNF, utilizando las herramientas JFlex y Cup. Este análisis se realiza de manera modular y extensible, respetando las mejores prácticas de desarrollo y manejo de proyectos en Java. A continuación, se detalla el diseño técnico y las decisiones adoptadas.

2.1. Estructura del Proyecto

El proyecto sigue una estructura modular que facilita la organización y mantenimiento del código. Los principales componentes del sistema son:

- **Clase Main:** Es el punto de entrada del programa. Gestiona la entrada de archivos, instancia el analizador léxico (`Lexer`) y procesa los tokens generados, además de imprimir la tabla de símbolos en un archivo de salida.
- **Analizador Léxico:** Implementado con JFlex, genera los tokens correspondientes a partir del código fuente de entrada. Este analizador también se encarga de manejar errores léxicos, registrándolos en la tabla de símbolos y permitiendo la recuperación en modo pánico.
- **Tabla de Símbolos:** Implementada en la clase `SymbolTable`, utiliza un mapa que asocia identificadores con listas de información (`SymbolInfo`), almacenando detalles como tipo, línea, columna y valor.
- **Gestión de Tokens:** La clase `sym`, generada por Cup, define los tokens utilizados en el análisis.

2.2. Herramientas y Tecnologías Utilizadas

- **JFlex:** Utilizado para generar el analizador léxico a partir de un archivo `.flex`. Este archivo contiene las definiciones de patrones regulares para los tokens, así como reglas para manejar errores léxicos.
- **JavaCup:** Integrado con JFlex mediante la directiva `%cup`, permite la generación de clases `Symbol` y `sym`, necesarias para el análisis.

- **Gradle:** Herramienta de automatización utilizada para compilar el proyecto, generar los archivos `.java` a partir de JFlex y Cup, y empaquetar el proyecto en un archivo JAR ejecutable. Esto asegura la reproducibilidad y simplifica la gestión de dependencias.

2.3. Flujo de Ejecución

1. **Lectura del Archivo:** El programa recibe el nombre de un archivo fuente como parámetro de entrada. Utiliza un `InputStream` para acceder a los archivos incluidos en el JAR.

2. **Análisis Léxico:** El `Lexer` generado por JFlex procesa el contenido del archivo, identificando tokens y registrándolos en la tabla de símbolos.

3. **Gestión de Errores:** Los errores léxicos son detectados y reportados en la consola. Además, se continúa el análisis para garantizar un procesamiento completo.

4. **Salida:** La tabla de símbolos generada se escribe en un archivo `output.txt`, detallando los identificadores, sus tipos, y la ubicación (línea y columna) en el código fuente.

2.4. Decisiones de Diseño

- **Modularidad:** Cada componente del sistema está encapsulado en clases específicas para facilitar la extensibilidad y el mantenimiento.

- **Manejo de Errores:** Se utiliza la técnica de recuperación en modo pánico en el analizador léxico para garantizar que el análisis continúe a pesar de los errores encontrados.

- **Automatización:** Gradle se emplea para la compilación y generación del JAR, permitiendo un flujo de trabajo eficiente y simplificado.

2.5. Generación del JAR

Gradle se configuró para compilar el proyecto, ejecutar JFlex para generar el analizador léxico y empaquetar todo en un archivo JAR ejecutable.

Esto se logra mediante la configuración del archivo `build.gradle`, que incluye tareas específicas para la integración de JFlex y Cup.

En resumen, este diseño garantiza un sistema robusto, extensible y fácilmente integrable con futuras fases del compilador, cumpliendo con los objetivos planteados para la fase de análisis léxico.

3. Librerías Usadas

En el desarrollo del proyecto se han utilizado varias librerías que facilitan la implementación y gestión de las funcionalidades requeridas. A continuación, se describen las principales librerías empleadas, sus propósitos y cómo contribuyen al análisis léxico y la generación de la tabla de símbolos.

3.1. Java Standard Library

La biblioteca estándar de Java proporciona las clases fundamentales utilizadas en la implementación del programa:

- `java.io.FileReader` y `java.io.InputStreamReader`: Estas clases permiten la lectura de archivos de entrada en diferentes formatos. Se usan para acceder y procesar el código fuente a analizar.
- `java.io.FileWriter`: Permite la creación y escritura de archivos de salida, como `output.txt`, donde se registra la tabla de símbolos generada.
- `java.util.Map` y `java.util.List`: Utilizadas para implementar la tabla de símbolos (`SymbolTable`), que asocia identificadores con su información relevante como tipo, línea, columna y valor.
- `java.nio.charset.StandardCharsets`: Facilita la especificación del conjunto de caracteres UTF-8 para garantizar la compatibilidad con archivos fuente que incluyen caracteres especiales.

3.2. JFlex

JFlex es una herramienta de generación de analizadores léxicos en Java. Su uso en el proyecto incluye:

- Definición de patrones regulares para identificar tokens del lenguaje, como palabras clave, operadores y literales.
- Manejo de errores léxicos mediante reglas específicas para caracteres no reconocidos.
- Integración con JavaCup mediante la directiva `%cup`, lo que permite la generación de tokens compatibles con el análisis sintáctico.

3.3. JavaCup

JavaCup es una herramienta para la generación de analizadores sintácticos. Aunque en esta fase del proyecto solo se utiliza para definir y gestionar tokens (`sym`) y estructuras (`Symbol`), su integración es esencial para futuras fases del compilador.

3.4. Librerías Personalizadas

Se han creado las siguientes clases específicas para el proyecto:

- `SymbolTable`: Implementa una tabla de símbolos utilizando `java.util.Map`, con métodos para agregar, buscar y verificar identificadores.
- `SymbolInfo`: Representa la información asociada a cada símbolo, incluyendo su tipo, posición (línea y columna) y valor.

3.5. Gradle

Se utilizó Gradle como herramienta de automatización para la construcción del proyecto. Las tareas específicas incluidas fueron:

- Compilación de los archivos fuente (`.java`).
- Generación del analizador léxico a partir del archivo `.flex`.
- Empaquetado del proyecto en un archivo JAR ejecutable.

3.6. Compatibilidad y Portabilidad

Todas las librerías empleadas son estándar o de código abierto, garantizando la compatibilidad en cualquier entorno que soporte Java. La automatización con Gradle simplifica la configuración del entorno y asegura que los pasos de construcción puedan ser replicados fácilmente en diferentes sistemas.

4. Análisis de Resultados

El desarrollo del proyecto se centró en implementar la fase de análisis léxico para el lenguaje definido en la gramática BNF, cumpliendo con los objetivos establecidos. A continuación, se presenta un análisis de los resultados obtenidos, destacando los objetivos alcanzados, los no alcanzados y las razones correspondientes.

4.1. Objetivos Alcanzados

- **Implementación del analizador léxico:** Se logró generar un analizador léxico funcional utilizando JFlex. Este reconoce los tokens definidos en la gramática, como palabras clave, operadores, literales y delimitadores.

- **Manejo de errores léxicos:** El analizador implementa la técnica de recuperación en modo pánico, permitiendo detectar errores léxicos sin detener el procesamiento del código fuente. Los errores son reportados con su ubicación precisa (línea y columna).

- **Generación de tabla de símbolos:** Se implementó exitosamente una tabla de símbolos que almacena información detallada de los identificadores encontrados, incluyendo su tipo, ubicación y valor, en un archivo de salida (`output.txt`).

- **Automatización con Gradle:** Se configuró Gradle para compilar el proyecto, generar el analizador léxico a partir del archivo `.flex`, y empaquetar el proyecto en un archivo JAR ejecutable. Esto asegura una construcción reproducible y simplifica el despliegue.

4.2. Objetivos No Alcanzados

- **Profundización en autómatas:** Aunque el analizador léxico cumple su función, no se logró un entendimiento profundo de los autómatas subyacentes, ya que JFlex automatiza gran parte de esta tarea. La generación de los autómatas desde las expresiones regulares fue transparente para los desarrolladores, limitando el aprendizaje práctico en este aspecto.

4.3. Razones para los Objetivos No Alcanzados

- El uso de JFlex facilita enormemente la implementación del análisis léxico al automatizar la construcción de autómatas finitos deterministas (DFA). Aunque esta herramienta es eficiente y práctica, reduce la necesidad de comprender a fondo los principios teóricos y prácticos detrás de los autómatas.

4.4. Conclusión

El proyecto cumplió con los objetivos fundamentales para esta fase, demostrando que el análisis léxico del lenguaje propuesto es funcional y extensible. Aunque no se logró un entendimiento profundo de los autómatas debido a la automatización proporcionada por JFlex, esto no afectó la funcionalidad del sistema. Este aspecto queda como un área de estudio adicional para comprender mejor los fundamentos teóricos detrás del análisis léxico.

5. Bitácora de Commits

La siguiente tabla presenta un registro detallado de los commits realizados en el repositorio durante el desarrollo del proyecto. Se incluyen el identificador del commit, el autor, la fecha y una breve descripción del cambio.

Fecha	Commit	Descripción
2024-12-18		
	906048f	**DaniloDuque** – jar created on build/libs
	8c61471	**DaniloDuque** – Corregir algunas regex
	b0d91f8	**DaniloDuque** – Regex de Comentario Multilinea funcionando

Fecha	Commit	Descripción
2024-12-17		
	97a6a66	**DaniloDuque** – Falta corregir el test.txt
	e7aa826	**DaniloDuque** – Debugging
	60f5f88	**kyaki** – Fixed empty space and one line comments
	8ee1365	**DaniloDuque** – DEBUGGING
	552eeab	**kyaki** – Output to text file
	2eb1f46	**DaniloDuque** – Working errors
	01fb73e	**DaniloDuque** – Started reporting line and column
	6aec151	**DaniloDuque** – Print all entry values
	5a2e796	**kyaki** – Working lexer
	4c36059	**DaniloDuque** – Lexer compiling
	d0bed73	**kyaki** – Added symbol table classes
2024-12-16		
	0b8f554	**DaniloDuque** – Moved test.txt
	21c82f0	**DaniloDuque** – ...
	3cfb2b9	**DaniloDuque** – Started lexer
	7815157	**DaniloDuque** – Changed repo

La bitácora refleja el progreso continuo del desarrollo, destacando los aportes de cada colaborador y el enfoque iterativo para alcanzar los objetivos del proyecto.